

Progress Report: ThinkAgain AI

1. Project Title

ThinkAgain AI

Team Name

MultiAgents

2. Project Objective

Develop an agent-based tutoring system that identifies a student's binary-search misconception, generates Socratic questions, evaluates the student's reasoning, and verifies understanding through a fresh transfer task.

3. Implemented Components

3.1 Diagnostic Agent

- Identifies the student's misconception from the submitted binary-search code.
- Uses a structured diagnostic schema.
- Records:
 - Misconception identifier
 - Confidence score
 - Evidence
 - Reasoning pattern
- The implemented diagnostic identified the misconception as `M1_INCOMPLETE_ELIMINATION`.

3.2 Socratic Agent

- Generates one Socratic question at a time.
- Uses different questioning angles to examine the student's reasoning.
- Implemented questioning angles include:
 - Eliminated range proof
 - Counterexample array

- Invariant restatement
- Opposite branch transfer

3.3 Evaluator Agent

- Evaluates the student's response to a Socratic question or transfer task.
- Produces a structured result:
 - PASS
 - REINFORCE
 - UNCERTAIN
- Records confidence, evidence, and reasoning assessment.

3.4 Transfer Agent

- Generates a fresh binary-search reasoning task.
- Checks whether the student can apply the corrected concept in a new context.
- Uses a structured transfer-task schema containing:
 - Task identifier
 - Prompt
 - Target reasoning

3.5 Learning State Tracking

The system maintains the student's learning state using:

- Misconception
- Current status
- Successful Socratic angles
- Reinforced Socratic angles
- Transfer result
- Recommended next action

3.6 Deterministic State Machine

The workflow is controlled through explicit states:

- DRAFTING
- GATING

- `AWAITING_EXPERT`
- `EVALUATING`
- `COMPLETE`
- `FAILED`

The state machine controls sequencing between stages, while the language model performs judgement within individual stages.

3.7 Persistent Storage

The system uses SQLite for persistent run storage.

Implemented storage features:

- Run creation and state tracking
- Append-only version history
- Record retrieval by type
- Complete run replay
- Token counters
- Step-attempt counters
- Human question storage
- Resume support

Database triggers prevent updates and deletions from the version history.

3.8 Budget and Failure Handling

The system includes:

- Per-step attempt limits
- Per-run token limits
- Persistent counters
- Budget checks before model calls
- Model fallback support
- API error classification
- Explicit failure recording
- Maximum state-machine step protection

3.9 Human-in-the-Loop Expert Interface

A reusable FastAPI-based expert callback interface is implemented. The current binary-search boundary-loop CLI still collects student responses in the terminal; the web callback is not yet the complete input path for this demo.

Features include:

- Displaying pending questions
- Opening a specific question
- Submitting an expert response
- Recording expert evidence separately
- Preventing repeated overwriting of answers
- Resuming the related run after an answer

4. Implemented Execution Workflow

The implemented boundary-loop workflow follows these stages:

1. Accept the student's binary-search attempt.
2. Diagnose the student's reasoning.
3. Identify the target misconception.
4. Generate a Socratic question.
5. Wait for the student's response.
6. Evaluate the response.
7. Update the learning state.
8. Repeat with a different Socratic angle when reinforcement is required.
9. Generate a fresh transfer task after a successful Socratic response.
10. Evaluate the transfer response.
11. Mark the learning state as transfer-passed or requiring further reinforcement.
12. Complete the run and preserve the complete history.

5. Execution Evidence

5.1 Initial Student Attempt

The demonstrated student attempt used:

```
else if (target > nums[mid]) {  
    left++;  
} else {  
    right--;  
}
```

This implementation retained `mid` in the active search interval after the midpoint had already been proven not to contain the target.

5.2 Diagnostic Result

- **Misconception:** `M1_INCOMPLETE_ELIMINATION`
- **Result:** The student understood which direction the search should move but did not initially exclude the midpoint itself from the next search range.
- **Risk identified:** Retaining `mid` can prevent the search interval from shrinking and can cause an infinite loop.

The confidence value is produced by the model for each run; `0.95` is an observed run result, not a hard-coded guarantee.

5.3 Socratic Evaluation Results

The following table records one observed execution, not a deterministic result guaranteed for every live model run:

Socratic Angle	Outcome	Confidence
Eliminated Range Proof	REINFORCE	0.95
Counterexample Array	REINFORCE	0.95
Invariant Restatement	REINFORCE	0.95
Opposite Branch Transfer	PASS	0.95

The first three questions reinforced the identified misconception. The fourth question demonstrated that the student understood:

- When `arr[mid] > target`, indices from `mid` onward are impossible.
- When `arr[mid] < target`, indices up to and including `mid` are impossible.

- The updated boundary must exclude the proven-invalid midpoint.

5.4 Transfer Task Result

The transfer task used the following values:

```
left = 2
right = 10
mid = 6
nums[mid] = 31
target = 14
```

The student correctly identified that indices 6 through 10 could be eliminated and set:

```
right = mid - 1 = 5
```

Transfer evaluation for the observed run:

- **Outcome:** PASS
- **Confidence:** 0.95
- **Transfer passed:** True

6. Final Learning State

```
Misconception: M1_INCOMPLETE_ELIMINATION
Status: TRANSFER_PASSED
Transfer passed: True
```

Successful Socratic angles:

- ELIMINATED_RANGE_PROOF
- COUNTEREXAMPLE_ARRAY
- INVARIANT_RESTATEMENT
- OPPOSITE_BRANCH_TRANSFER

Recommended next action:

Use a related binary-search boundary problem in the next encounter.

7. Run Completion

Final state: complete
Boundary Loop completed.

The run history was recorded and can be replayed using:

```
python scripts/smoke.py replay <run_id>
```

The actual run ID is printed when the run starts and is different for each run.

8. Current Progress Summary

- Diagnostic agent implemented.
- Socratic questioning agent implemented.
- Evaluation agent implemented.
- Transfer-task agent implemented.
- Structured Pydantic schemas implemented.
- Deterministic state-machine workflow implemented.
- Persistent SQLite storage implemented.
- Append-only history enforcement implemented.
- Budget and attempt controls implemented.
- Model fallback and failure handling implemented.
- Human expert callback interface implemented.
- Binary-search misconception workflow executed successfully.
- Student reasoning progressed from repeated reinforcement to a successful transfer evaluation.

9. How to Run

From the `MultiAgents` directory, install the dependencies and run the live boundary-loop demo with:

```
python -m pip install -r requirements.txt  
python run_boundary_loop.py
```

This command requires `OPENROUTER_API_KEY` in `.env` or in the process environment. It asks for the student's binary-search attempt and responses; finish each multiline input by typing `END` on its own line. For an offline workflow check with canned responses and no API key, run:

```
python scripts/smoke.py run --stub
```